
LECTURE NOTES

Bayesian Learning

AI512 — Introduction to Machine Learning

Simon Holm
2026-09-04



DEPARTMENT OF MATHEMATICS
AND COMPUTER SCIENCE

Contents

1	Maximum Likelihood Estimation (MLE)	3
1.0.1	Example	3
2	Bayesian Learning	4
3	Maximum A-Posteriori Estimation	7
4	Monte Carlo Integration	8
5	Generative Models	8
6	Naive Bayes Classifier	9

1 Maximum Likelihood Estimation (MLE)

MLE assumes that there is even more uncertainty and randomness than previously covered. MLE is like rewinding the data-generating process and asking which parameters could have most plausibly produced what we observed. (So we can replicate them)

Let

$$S = \{x_1, x_2, \dots, x_m\}$$

At the stage of modeling, we draw a hypothesis about how this data could have been generated. Let our hypothesis be that the data is generated as independent samples a distribution following a parametric density function $p(x | \theta)$, where θ is the parameter of the distribution. Then the probability density function of the random variable S representing the occurrence of the dataset is given by

$$p(S | \theta) = p(x_1, x_2, \dots, x_m | \theta) = \prod_{i=1}^m p(x_i | \theta).$$

If we find θ we can predict new data in the same shape as S

The expression above is a density function and is called the **likelihood** of parameters θ given the dataset S .

We can do this by maximizing the likelihood function $p(S | \theta)$.

This uses “log-likelihood”. Where because for $p > 0$, $\log(\prod(P) = \sum(\log(p)))$ This way we can inspect each p . So

$$\theta_{MLE} = \arg \max_{\theta} \sum_{i=1}^m \log p(x_i | \theta).$$

1.0.1 Example

Let us give an example. Choose $p(x | \theta) = \mathcal{N}(x | \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$, where $\theta = (\mu, \sigma^2)$.

Then consider that a single term of the log-likelihood function for a data point x_i is given by

$$\log p(x_i | \theta) = -\frac{1}{2} \log(2\pi\sigma^2) - \frac{(x_i - \mu)^2}{2\sigma^2}.$$

We can then maximize the likelihood with respect to μ and σ^2 by setting the gradient of the full likelihood function to zero and solving for μ .

$$\frac{\partial}{\partial \mu} \sum_{i=1}^m \log p(x_i | \theta) = \sum_{i=1}^m \frac{x_i - \mu}{\sigma^2} = 0 \implies \mu_{MLE} = \frac{1}{m} \sum_{i=1}^m x_i.$$

We can also solve for σ^2 .

$$\frac{\partial}{\partial \sigma^2} \sum_{i=1}^m \log p(x_i | \theta) = -\frac{m}{2\sigma^2} + \frac{1}{2\sigma^4} \sum_{i=1}^m (x_i - \mu)^2 = 0 \implies \sigma_{MLE}^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu)^2.$$

Note that the parameters for a normal distribution is just the mean μ and variance σ^2

Now that we have found the parameters that fits the data μ_{MLE} and σ_{MLE}^2

Given a test input x_* , the learned model predicts the output in the form of a distribution, which is called the **predictive distribution**:

$$y_* | x \sim \mathcal{N}(y_* | w_{MLE}^T \phi(x_*), \sigma_{MLE}^2)$$

This determines how likely an output is given an input. If the likelihood is good, when we can use

$$\hat{y} = w_{MLE}^T \phi(x_*)$$

If we have a safety-critical prediction task, we can also build confidence sets. For instance,

$$\begin{aligned} P(|y_* - w_{MLE}^T \phi(x_*)| \leq \sigma_{MLE}) &= P(w_{MLE}^T \phi(x_*) - \sigma_{MLE} \leq y_* \leq w_{MLE}^T \phi(x_*) + \sigma_{MLE}) \\ &= \int_{w_{MLE}^T \phi(x_*) - \sigma_{MLE}}^{w_{MLE}^T \phi(x_*) + \sigma_{MLE}} \mathcal{N}(y_* | w_{MLE}^T \phi(x_*), \sigma_{MLE}^2) dy_* \\ &= \int_{w_{MLE}^T \phi(x_*) - \sigma_{MLE}}^{w_{MLE}^T \phi(x_*) + \sigma_{MLE}} \frac{1}{\sqrt{2\pi\sigma_{MLE}^2}} \exp\left(-\frac{(y_* - w_{MLE}^T \phi(x_*))^2}{2\sigma_{MLE}^2}\right) dy_* \\ &= \int_{-1}^1 \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{z^2}{2}\right) dz = 0.68. \end{aligned}$$

2 Bayesian Learning

The MLE approach cannot model the potential dependency of the variance on the input x . That means that because σ^2 is a single number. The problem with having a **single variance** (σ^2) is that it **assumes all inputs have the same uncertainty**, which is often unrealistic. Let me explain carefully.

The only source of uncertainty is on the precise values of the model parameters, which can be mitigated by collecting more data due to the law of large numbers. This ansatz is called **frequentist learning**.

In real-world problems, the modeler is never certain about the model and in many cases they are interested in accounting for this source of uncertainty.

Bayesian learning offers a way to do this. Let us see how it works on the same linear regression problem.

$$w \sim \mathcal{N}(w \mid 0, \alpha^{-1}I)$$

$$y_i \mid x_i, w \sim \mathcal{N}(y_i \mid w^T \phi(x_i), \sigma^2), \quad i = 1, \dots, m.$$

This time we are not assuming that w (the distribution) is a fixed parameter, but a random variable. We start with the prior belief that $w \sim p(w)$. Based on the data, we update our belief about w to $p(w|S)$.

- $p(w)$ is called the **prior distribution**
- and $p(w|S)$ is called the **posterior distribution**.

The goal of **learning** in Bayesian machine learning is to calculate the posterior distribution $p(w|S)$ given the dataset S . This means that we start with a “unbiased” distribution and update it as we gather more information

This is done via the Bayes’ rule:

$$p(w \mid S) = \frac{p(S \mid w)p(w)}{p(S)} = \frac{p(S \mid w)p(w)}{\int p(S \mid w)p(w)dw}.$$

The term in the denominator $p(S) = \int p(S \mid w)p(w)dw$ is called the **evidence** (also called the **marginal likelihood**). This means “Given our model (all possible parameter settings), how likely is it that we would see the data we actually observed?”

- In almost every real-world use case, the evidence is hard to calculate because of complexity. Therefore, we will not be able to calculate the posterior distribution exactly. Instead, we will use approximations. The whole field of Bayesian machine learning is about finding good approximations to the posterior distribution.
- The evidence quantifies the fit of the whole model family to data, as it can be viewed as the average likelihood with respect to the prior distribution. Therefore, it can be used for model selection. For instance, if we have two competing models $p(S, w)$ and $p'(S, w')$, we can choose the one with the higher evidence.

Let us next calculate the posterior distribution for the linear regression problem. We have:

$$\begin{aligned}
p(w \mid S) &= \prod_{i=1}^m \mathcal{N}(y_i \mid w^T \phi(x_i), \sigma^2) \mathcal{N}(w \mid 0, \alpha^{-1} I) \\
&= \mathcal{N}(w \mid 0, \alpha^{-1} I) \prod_{i=1}^m \mathcal{N}(y_i \mid w^T \phi(x_i), \sigma^2) \\
&\propto \exp\left(-\frac{1}{2} w^T \alpha I w\right) \prod_{i=1}^m \exp\left(-\frac{1}{2\sigma^2} (y_i - w^T \phi(x_i))^2\right) \\
&\propto \exp\left(-\frac{1}{2} w^T \alpha I w - \frac{1}{2\sigma^2} \sum_{i=1}^m (y_i - w^T \phi(x_i))^2\right) \\
&\propto \exp\left(-\frac{1}{2} w^T \left(\alpha I + \frac{1}{\sigma^2} \sum_{i=1}^m \phi(x_i) \phi(x_i)^T\right) w + \frac{1}{\sigma^2} \sum_{i=1}^m y_i \phi(x_i)^T w\right) \\
&= \mathcal{N}\left(w \mid \left(\alpha I + \frac{1}{\sigma^2} \sum_{i=1}^m \phi(x_i) \phi(x_i)^T\right)^{-1} \frac{1}{\sigma^2} \sum_{i=1}^m y_i \phi(x_i), \left(\alpha I + \frac{1}{\sigma^2} \sum_{i=1}^m \phi(x_i) \phi(x_i)^T\right)^{-1}\right).
\end{aligned}$$

A neater notation would be possible considering that the posterior mean calculation involves the inverse of the posterior covariance.

$$p(w \mid S) = \mathcal{N}(w \mid \mu_{post}, \Sigma_{post}),$$

where

$$\begin{aligned}
\Sigma_{post} &= \left(\alpha I + \frac{1}{\sigma^2} \sum_{i=1}^m \phi(x_i) \phi(x_i)^T\right)^{-1} \\
\mu_{post} &= \Sigma_{post} \left(\frac{1}{\sigma^2} \sum_{i=1}^m y_i \phi(x_i)\right).
\end{aligned}$$

For a new test input x_* , the Bayesian model predicts the output in the form of a distribution:

$$\begin{aligned}
p(y_* \mid S, x_*) &= \int p(y_* \mid x_*, w) p(w \mid S) dw \\
&= \int \mathcal{N}(y_* \mid w^T \phi(x_*), \sigma^2) \mathcal{N}(w \mid \mu_{post}, \Sigma_{post}) dw \\
&= \mathcal{N}(y_* \mid \mu_{post}^T \phi(x_*), \sigma^2 + \phi(x_*)^T \Sigma_{post} \phi(x_*)).
\end{aligned}$$

The resulting distribution $p(y_* \mid S, x_*)$ is called the **posterior predictive distribution**. This distribution has some remarkable properties:

- It considers all possible values of w and averages over them proportionally to their posterior probability. This essential and unique property of Bayesian models is called **model averaging**. It is a way of accounting for model uncertainty.
- Its variance depends on the input x_* . Hence, it is able to take into account potential changes in the model confidence in different regions of the input space. This property of a probabilistic model is called **heteroscedasticity**. The Bayesian approach provides this property for free, whereas the frequentist approach requires a separate model for the variance.

In the cases where the predictor is a distribution, there are multiple ways one can use it to make a final prediction. For instance, one can choose the mode of the predictive distribution:

$$\hat{y} = \arg \max_y p(y_* = y \mid S, x_*).$$

This is called the **Bayes predictor**. One can alternatively choose to take a sample from the predictive distribution:

$$\hat{y} \sim p(y_* \mid S, x_*).$$

This is called the **Gibbs predictor**. Bayes predictor is known to be the optimal predictor in terms of the expected loss. (Example from lectures of predicted)

3 Maximum A-Posteriori Estimation

Much like in Bayesian Linear Regression, with **MAP** we can estimate the **posterior mean** of the weight distribution:

$$w_{\text{MAP}} = \arg \max_w p(w \mid S) = \arg \max_w p(S \mid w) p(w) = \arg \max_w \log p(S \mid w) + \log p(w)$$

The intractable denominator $p(S)$ drops out since it does not depend on w . This makes the training objective tractable.

In Bayesian linear regression, the posterior is Gaussian:

$$p(w \mid S) = \mathcal{N}(w \mid \mu_w, \Sigma_w)$$

where the posterior covariance Σ_w is fully determined by the prior and the data. Once Σ_w is known, the MAP estimate is simply the **posterior mean**:

$$w_{\text{MAP}} = \mu_w = \Sigma_w \left(\frac{1}{\sigma^2} \sum_{i=1}^m y_i \phi(x_i) \right)$$

For a given test input x^* , the MAP estimate predicts the output as:

$$y_* \mid x_* \sim p(y_* \mid w_{\text{MAP}}, x_*)$$

Since the MAP estimate is a **point estimate**, it does not capture the uncertainty in the model parameters. Once you have Σ_w , computing MAP is just **matrix multiplication**.

Because of this, it is not considered a fully Bayesian approach, as it cannot perform model averaging over the posterior.

To do model averaging, you would need the average predictions over the posterior including all its uncertainties.

MAP **ignores this uncertainty**, so it can't capture:

- Variance in predictions due to uncertain parameters
- Risk of overconfident predictions if data is scarce
- True Bayesian model averaging benefits

4 Monte Carlo Integration

Consider an integral of the form:

$$\mathbb{E}[f(x)] = I = \int f(x)p(x) dx.$$

for some function $f(x)$ and probability density function $p(x)$. If we can take m samples from $p(x)$, we can approximate the integral by the sample average:

$$I \approx \frac{1}{m} \sum_{i=1}^m f(x_i),$$

where $x_i \sim p(x)$. This is called the **Monte Carlo integration**.

5 Generative Models

For a supervised learning problem where the data comes from $x, y \sim D$ for an unknown data distribution D . We can approach the modeling problem in two ways based on what we want to approximate about the data distribution. Consider that the joint distribution factorizes in two ways: $p(x, y) = p(y|x)p(x)$

$$p(x, y) = p(x|y)p(y)$$

We can choose the first factorization and account for $p(x)$ using Monte Carlo integration based on the training samples. Then it suffices to approximate $p(y|x)$. This approach is called **discriminative modeling**. The probabilistic and Bayesian linear regressors we developed above were all discriminative models since they fitted $p(y|w, x)$ to data.

An alternative approach would be to choose the second factorization: $p(x, y) = p(x|y)p(y)$ and aim to approximate both $p(y)$ and $p(x|y)$. Notice that this approach attempts to infer the whole data generating process, where a label is first created and the related input is generated. A real-world example could be a person choosing the digit to draw first and then

drawing it. The picture of the resulting drawing because the input of the aimed classifier. This approach is called **generative modeling**.

For instance, we can fit a generative model on a data set $S = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$ by choosing a parametric distribution family $p(x | y, \theta)$. For a classification problem, we can approximate $p(y)$ by class frequencies, i.e.

$$P(y = c) = \frac{1}{m} \sum_{i=1}^m \mathbb{1}(y_i = c).$$

and choose the class-conditional distribution to be:

$$p(x | y = c, \theta_c) = p(x | \theta_c),$$

that is, each class shares the same distribution family but has its own parameters. Then we can fit the parameters θ_c on the training samples belonging to class c by maximum likelihood estimation:

$$\theta_c^{MLE} = \arg \max_{\theta_c} \sum_{\{i: y_i = c\}} \log p(x_i | \theta_c).$$

Given a test input x_* , the generative model predicts the output in the form of a distribution:

$$p(y = c | x_*) = \frac{p(x_* | y = c, \theta_c) P(y = c)}{\sum_{c'} p(x_* | y = c', \theta_{c'}) P(y = c')}.$$

6 Naive Bayes Classifier

In the generative classification example above, consider the case that the class-conditionals are normal distributions with full covariance matrices:

$$p(x | y = c, \theta_c) = \mathcal{N}(x | \mu_c, \Sigma_c).$$

Then the training set will be used to fit $d + \frac{d(d+1)}{2}$ parameters per class, where d is the dimensionality of the input. The first term is due to the parameters of the mean and the rest is due to the parameters of the covariance matrix. This is a lot of parameters to fit. We can instead make the simplifying assumption that the class-conditionals of the individual features are independent. Then we have:

$$\begin{aligned} p(x | y = c, \theta_c) &= \prod_{j=1}^d p(x_j | \theta_{j,c}) \\ &= \prod_{j=1}^d \mathcal{N}(x_j | \mu_{j,c}, \sigma_{j,c}^2). \end{aligned}$$

The assumption that the class-conditionals factorize across individual features is called the **naive Bayes assumption**. For the normal distributed class-conditionals used in our

example, the resulting model will then have only $2d$ parameters per class. The resulting generative classifier is called a **naive Bayes classifier**. Applied to normal distributed class-conditionals, the naive Bayes classifier follows the same formulas as above with the assumption that the covariance matrix is diagonal.

$$P(x_1 = a, x_2 = b | y = c) = \frac{\#(x_1=a) \wedge \#(y=c)}{\#(y=c)} \cdot \frac{\#(x_2=b) \wedge \#(y=c)}{\#(y=c)}$$

$$P(\text{underbrace}(y = c^*, \text{"prediction"}) | \text{underbrace}((x_1 = a^*, x_2 = b^*), \text{"queryinput"})) = (P(x_1 = a^* | y = c^*) \cdot P(x_2 = b^* | y = c^*) \cdot p(y = c^*)) / (\sum_{c^* \in \text{underbrace}(Y, \{y_1\})} P(x_1 = a^* | p = c^*) \cdot P(x_2 = b^* | p = c^*) \cdot P(p = c^*))$$